

Why Prefer Iteration?

Performance: Iteration is more efficient for generating long sequences because it avoids the overhead associated with recursive calls, such as pushing and popping function calls onto the stack.

Memory Usage: Iteration uses less memory compared to recursion, especially for large sequences, as it doesn't require storing intermediate results on the call stack.

Implementation

We'll encapsulate the Fibonacci generation logic within a service class, adhering to Laravel's best practices for organizing business logic.

Step 1: Create the FibonacciGenerator Service Class

```
php artisan make:service FibonacciGenerator
```

This command creates a `FibonacciGenerator.php` file in the `app\Services` directory.

Step 2: Implement the Fibonacci Sequence Generation Logic

Open the `FibonacciGenerator.php` file and implement the logic to generate a Fibonacci sequence iteratively.

```
<?php

namespace App\Services;

class FibonacciGenerator
{
    /**
     * Generates a Fibonacci sequence up to the nth term.
     *
     * @param int $length The length of the Fibonacci
     *                    sequence to generate.
     * @return array The generated Fibonacci sequence.
     */
    public function generateSequence(int $length): array
    {
        $sequence = [0, 1]; // Start with the first two terms of
                           // the Fibonacci sequence

        for ($i = 2; $i < $length; $i++) {
            $nextTerm = $sequence[$i - 1] + $sequence[$i - 2];
            $sequence[] = $nextTerm;
        }

        return $sequence;
    }
}
```

Step 3: Use the Service in a Controller

Inject the FibonacciGenerator service into a controller to utilize its functionality.

```
<?php

namespace App\Http\Controllers;

use App\Services\FibonacciGenerator;

class FibonacciController extends Controller
{
    protected $fibonacciGenerator;

    public function __construct(FibonacciGenerator
                                $fibonacciGenerator)
    {
        $this->fibonacciGenerator = $fibonacciGenerator;
    }

    public function showFibonacciSequence()
    {
        $length = 10; // Specify the length of the Fibonacci
                       sequence
        $sequence = $this->fibonacciGenerator-
                    >generateSequence($length);

        return view('fibonacci.sequence', ['sequence' =>
                                           $sequence]);
    }
}
```

Step 4: Display the Sequence in a View

Create a Blade template to display the generated Fibonacci sequence.

```
<!-- resources/views/fibonacci/sequence.blade.php -->

<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>Fibonacci Sequence</title>
    </head>
    <body>
        <h1>Fibonacci Sequence</h1>
        <ul>
            @foreach($sequence as $term)
                <li>{{ $term }}</li>
            @endforeach
        </ul>
    </body>
</html>
```

This implementation uses an iterative approach to generate the Fibonacci sequence, which is more efficient and suitable for generating long sequences